Um Modelo para Gerenciamento de Perfis de Entidades Através de Inferência em Trilhas

André Wagner

1 Introdução

Atualmente, as aplicações têm se concentrado na coleta de informações a respeito dos usuários, com o objetivo de permitir que elas possam conhecer suas preferências, possibilitando a oferta de serviços personalizados. Assim, estas aplicações organizam seu conhecimento a respeito do usuário em *perfis de usuário*, onde são salvas as informações em um formato baseado em um *modelo de usuário* específico.[1]

Sistemas sensíveis a contexto são aplicações nas quais o contexto atual de um usuário é levado em consideração na disponibilização de serviços. Dey[2] define contexto como qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade, onde uma entidade pode ser uma pessoa, local ou objeto considerado relevante à interação entre o usuário e a aplicação. Desta forma, o *perfil* torna-se parte essencial na descoberta de informações pertinentes aos usuários que possam ser utilizadas na construção de um contexto.

Os ambientes de *computação ubíqua* são parte de um cenário na qual serviços e informações estão disponíveis a qualquer momento, em qualquer lugar[3]. Esta visão vem sendo potencializada pela diminuição do custo e aumento da capacidade de memória e processamento dos dispositivos computacionais, bem como pela cada vez maior disponibilidade de recursos de localização e conectividade. A computação ubíqua enfatiza também a "tecnologia calma", onde a sobrecarga de informação é substituída pela disponibilização de recursos de forma inteligente, tomando a atenção do usuário apenas nos momentos em que é mais relevante.[4]

A capacidade de uma aplicação compreender o contexto de uma entidade pode ser amplificada através do uso de *trilhas*[5]. Uma trilha é o histórico de contextos visitados por uma entidade, e das ações realizadas pela entidade no contexto.

Embora existam alguns trabalhos que utilizam trilhas como forma de armazenar e processar informações de um usuário[6, 5], as trilhas são utilizadas com o objetivo de aquisição de informações para suprir uma necessidade específica do modelo.

Este trabalho busca preencher esta lacuna apresentando um modelo para gerenciamento de perfis, através de inferência em trilhas. O objetivo do modelo é permitir que qualquer aplicação que faça uso de trilhas possa ser capaz de montar seu próprio modelo de perfil e definir as regras necessárias para que este perfil seja inferido a partir das trilhas.

A Seção 2 apresenta conceitos gerais a respeito de gerenciamento de perfis, e na Seção ?? são comparados trabalhos relacionados. A Seção 3 apresenta o modelo, e a Seção ?? conclui comparando o modelo apresentado com os relacionados, e traçando possíveis trabalhos futuros.

2 Conceitos Gerais

A coleta de informações de um usuário pode ser *explícita* ou *implícita*[1]. Na coleta *explícita*, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta *implícita*, a informação é inferida a partir do estudo do comportamento dos usuários.

Informações a respeito de um usuário colhidas por uma aplicação podem ser úteis para outras aplicações que tenham o mesmo usuário [1]. No entanto, os diferentes *modelos de usuário* utilizados pelas aplicações tornam difícil a utilização de informações de um mesmo *perfil de usuário* de uma aplicação por outra. Além disso, algumas das informações do usuário podem ser válidas apenas dentro de um *contexto* específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Visando resolver estas questões, o conceito de *personalização inter-sistemas* foi introduzido [7] como forma de compartilhar informações de usuários entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de *sistemas de modelos de usuário* é o principal desafio da pesquisa atual na área de personalização.[1]

O IEEE Standard Computer Dictionary [8] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizar a informação que foi trocada." No âmbito da personalização, interoperabilidade é a capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogênias e integrar esta informação em um modelo de usuário de granularidade própria[9]. Os dois principais modelos de interoperabilidade são: (i) *interoperabilidade sintática*, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação, e (ii) *interoperabilidade semântica*, onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de usuários interoperáveis são [1]:

Baseados em padrões : são baseados numa visão *top-down*, definindo um padrão que todas as aplicações envolvidas devem observar;

Baseados em mediação : se baseiam numa visão *bottom-up*, onde ocorre algum tipo de *reconciliação* entre os diferentes modelos de representação.

3 eProfile

O eProfile é um gerenciador de perfis voltado para ambientes distribuídos, que infere dados de perfil através da inferência das trilhas de uma entidade. A trilha é disponibilizada por um gerenciador de trilhas que se conecta ao eProfile, onde é feita uma mediação entre o formato de dados da trilha e do eProfile.

A figura tradicional do usuário na personalização é extendida aqui pelo conceito de *entidade*. Uma entidade é um conceito mais amplo que o de um usuário, não representando apenas pessoas mas também objetos que possam criar uma trilha e ter um perfil, como um veículo ou um dispositivo móvel. Uma entidade pode também ser composta por um grupo de entidades como, por exemplo, uma frota de caminhões onde cada caminhão tem seu perfil único, e onde a frota tem um perfil também.

A ontologia de perfil do eProfile pode ser extendida pelas aplicações, de modo que as aplicações possam armazenar dados de perfil que interessem apenas a si mesmos. Assim, as aplicações podem disponibilizar suas próprias regras para a inferência das informações relacionadas à sua própria estrutura de dados. Os dados inferidos ficam então disponíveis para serem consultados pelas entidades.

3.1 Modelo

A Figura 1 apresenta o modelo do eProfile.

As *aplicações* são os aplicativos que são utilizados pelas entidades para se conectar aos gerenciadores de trilhas e perfil. Neste modelo, a aplicação se conecta a um gerenciador de trilhas genérico, que é responsável pelo armazenamento da trilha da entidade. O perfil da entidade será inferido a partir das trilhas registradas no gerenciador.

As trilhas das entidades são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha (Figura 2), onde em cada um deles a trilha tem um formato específico, é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile, que será apresentado mais adiante. Este trabalho é realizado por um agente entitulado *conversor*, que é capaz de realizar a conversão para os formados de trilha suportados pelo eProfile.

Desta forma, o eProfile não tem a necessidade de conhecer a aplicação – mas precisa conhecer o formato utilizado pelo gerenciador de trilhas.

A trilha do usuário é então armazenada dentro do eProfile em uma *ontologia de trilha*. Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser extendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado. A forma como ocorre esta extensão é detalhada na Seção 3.2.

Cada aplicação tem a possibilidade de montar seu próprio perfil. Para que o eProfile processe este perfil, ele precisa de três informações: (i) a trilha da entidade, (ii) o modelo de perfil da aplicação

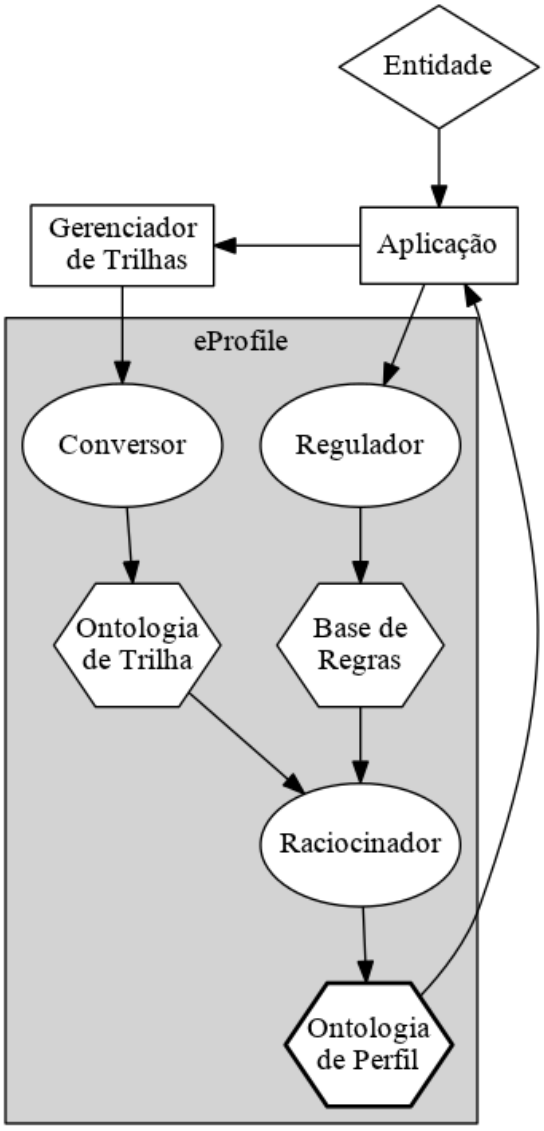

Figura 1: Modelo do eProfile, com as setas representando o fluxo de dados

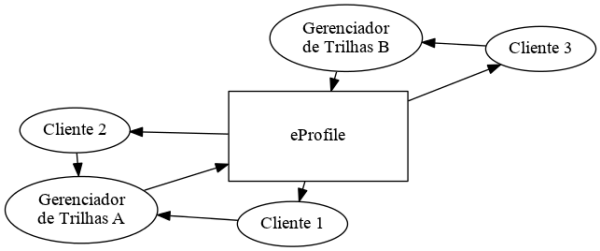

Figura 2: eProfile conectado a mais de um gerenciador de trilhas

e (iii) as regras de inferência para o perfil. A trilha da entidade é carregada pelo eProfile a partir do gerenciador de trilhas, enquanto as outras duas informações são enviadas diretamente da aplicação para o eProfile. Desta forma, as regras de inferência definidas pela aplicação serão utilizadas para inferir seu próprio perfil.

O agente responsável pela recepção das regras é o *regulador*. O regulador tem a função de receber as regras, validá-las e inserí-las na *base de regras*. Juntamente com as regras, é especificado a criticidade da informação. O sistema utilizará esta informação para determinar com que freqüência a inferência desta informação será executada.

Uma vez de posse das trilhas e das regras, o agente *raciocinador* realiza a inferência da trilha, gerando assim a *ontologia de perfil*. Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais as aplicações vão especializar, utilizando-as para armazenar informações que sejam úteis a eles. A aplicação pode então conectar-se ao eProfile e buscar o perfil de sua entidade. A Figura 1 exemplifica isso, onde a seta que conecta a Ontologia de Perfil (no eProfile) à aplicação mostra o sentido do fluxo de dados.

3.2 Ontologias

O eProfile utiliza internamente duas ontologias: uma ontologia de trilhas e uma ontologia de perfis. Para o armazenamento e processamento das ontologias, foram estudadas algumas linguagens de representação de conhecimento CycL[10], RDF/S[11], DAML+OIL[12] e OWL[12].

Devido à expressividade e ao suporte de ferramentas externas, o OWL foi escolhido como a lin-

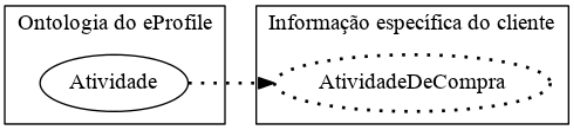

Figura 3: Exemplo da extensão de uma classe do eProfile

guagem de ontologias utilizado pelo eProfile para modelagem da ontologia do perfil.

As ontologias utilizadas pelo eProfile – tanto a ontologia de trilhas quanto a ontologia de perfil – são ontologias básicas, extendidas através de subclasses de acordo com o formato da trilha enviada pelo gerenciador (no caso da ontologia de trilha) ou das necessidades da aplicação (no caso da ontologia do perfil). Isto é representado na Figura 3, onde o eProfile oferece uma classe genérica "Evento", que é especializada pela classe "EventoDeCompra", utilizada por um dos gerenciadores de trilha.

3.2.1 Ontologia de Trilha

O eProfile implementa uma ontologia de trilha. Esta ontologia será alimentada a partir dos diferentes gerenciadores de trilhas suportados, onde o agente *conversor* é responsável por receber os dados no formato do gerenciador de trilhas e convertê-los para a ontologia de trilhas do eProfile. Assim, o raciocinador terá uma ontologia única sobre a qual poderá raciocinar.

A ontologia de trilha está representada na Figura 4, onde todos os relacionamentos, representados pelas setas, são do tipo "has-some" (tem-algum(s)).

Abowd e Dey definem que aplicações sensíveis a contexto estão interessadas em saber *quem*, *onde*, *quando* e *o quê* (isto é, o que o usuário está fazendo), de forma a poderem descobrir por que a situação está ocorrendo[13]. Assim, uma trilha é representada por um conjunto de eventos caracterizados pelas seguintes classes:

Indentity: É a identidade da entidade que está realizando a ação. Esta entidade pode estar realizando a ação com um ou mais recursos (*Resource*) (como um aplicativo ou dispositivo) e pode estar vinculada a um perfil (*Profile*) dentro do eProfile. A classe **Participants** contém as identidades das entidades que estão participando deste evento;

Location: Onde está ocorrendo a ação. Uma localização pode ser uma localização física específica (como uma sala) ou não (uma estrada, por exemplo). A localização pode estar vinculada a uma região (*Region* (que, adicionalmente, pode estar vinculada a outras subregiões) e geralmente possui uma localização global (GPS) vinculada a ela. Outros tipos de localização podem ser extendidas a partir da classe;

Time: Reponde à questão de *quando* ocorreu o evento;

Activity: Refere-se à ação que o usuário está executou (ou tentou executar). É a classe mais abrangente da ontologia, e a que possivelmente será a mais especializada pelos gerenciadores de trilha.

3.2.2 Ontologia de Perfil

A ontologia de perfil é a ontologia utilizada para armazenar as informações de perfil inferidas a partir da trilha. Esta ontologia está representada na Figura 5.

A ontologia possui duas classes básicas: *Entity* (entidade) e *Information* (informação). Assim, os perfis de várias entidades podem ser armazenados nesta ontologia, onde cada perfil possui um conjunto de informações. Este conjunto de informações será extendido pelas aplicações através de herança (subclasses).

Cada entidade é identificada por um nome único, mesmo que o perfil seja alimentado por várias aplicações. Desta forma, é possível que uma aplicação tenha acesso a informações de perfil gravadas por outra aplicação, desde que ela tenha conhecimento das classes utilizadas pelo outro.

As informações podem ser de três tipos:

PastInformation: refere-se a eventos que ocorreram no passado. As informações armazenadas nesta classe são, de um modo geral, cumulativas. Exemplos seriam as escolas onde uma pessoa estudou, ou os destinos onde um caminhão já entregou alguma carga;

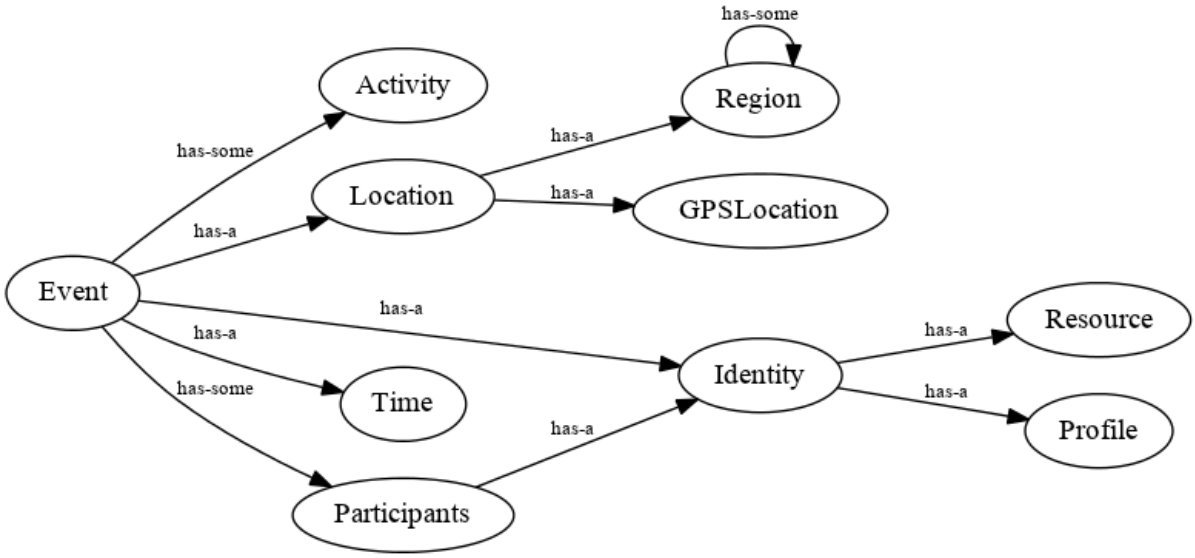

Figura 4: Ontologia de Trilha do eProfile

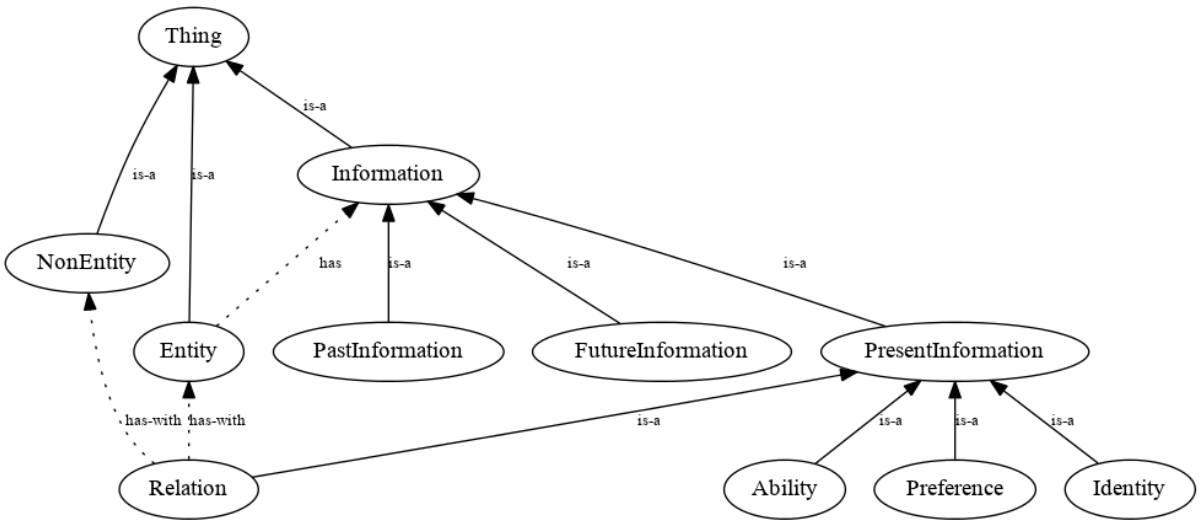

Figura 5: Ontologia de Perfil do eProfile

PresentInformation: são informações atuais da entidade, subdivididas em subclasses descritas mais adiante;

FutureInformation: são informações previstas a respeito do futuro de uma entidade, como o conjunto de disciplinas que uma pessoa tem a cursar, ou quantos quilômetros um caminhão pode percorrer antes de reabastecer.

As informações atuais podem ser dos seguintes tipos:

Identity: informações que identificam a entidade, como dados cadastrais.

Preference: preferências da entidade. Podem ser informações a respeito de interesses de um usuário ou aspectos mais técnicos, como protocolo preferencial de comunicação, ou até mesmo informações compostas, como qual qualidade de vídeo utilizar para cada banda de comunicação;

Ability: conjunto de capacidades de uma entidade. Capacidades incluem tanto habilidades de um usuário (como jogar xadrez) quanto capacidades de um serviço (por exemplo, suporte a webservices SOAP);

Relation: conjunto de relações relevantes desta entidade. As relações podem dizer respeito a uma outra entidade (por exemplo, um usuário que tenha outros amigos ou um carro que tem um dono) ou uma não-entidade (como um usuário que tem um dispositivo). No caso de relações com outras entidades, se a entidade possuir um perfil cadastrado, é possível realizar uma vinculação.

Todos as extensões (subclasses) da ontologia são definidas pelas aplicações, e todas as informações do perfil são inferidas. A forma como esta inferência ocorre é descrita na Seção 3.4.

3.3 Agentes

Como o eProfile é um modelo onde é necessário inferir informações de forma autônoma a respeito de dados armazenados em uma ontologia, a tecnologia de agentes autônomos foi escolhida para o desenvolvimento.

O modelo do eProfile é composto por três agentes: (i) conversor, (ii) regulador e raciocinador.

O **agente conversor** é responsável por realizar a conversão dos dados da trilha do gerenciador de trilhas (externo) para a ontologia de trilha do eProfile (interno). A trilha do gerenciador pode ser disponibilizada em qualquer formato e por qualquer meio, desde que o eProfile ofereça suporte ao gerenciador de trilhas específico. O agente conversor tem também a responsabilidade de verificar, a cada período pré-determinado, atualizações nos dados disponibilizados no gerenciador.

O **agente regulador** realiza o recebimento das regras enviadas pela aplicação. Uma vez que as regras sejam recebidas, elas são validadas e, se forem consideradas válidas, são armazenadas na base de regras. A aplicação é então informada do aceite ou rejeição. O agente pode rejeitar uma regra se: (i) a regra for inválida ou (ii) se for verificado que a execução da regra irá consumir recursos excessivos do servidor.

O **agente raciocinador** é responsável pela inferência das trilhas de acordo com as regras enviadas pela aplicação, e posterior atualização do perfil da entidade. A execução do agente não necessita que o cliente ou o gerenciador de trilhas esteja conectado, e ocorre da seguinte forma:

1. O agente verifica as regras, comparando a data da útlima execução da regra com a freqüência de execução, definida pela aplicação na criação da regra.

2. Caso seja o momento da execução da regra, a mesma é executada através da biblioteca externa HermiT[14].

3. Caso a regra tenha sido executada com sucesso, o resultado da regra é atualizado no perfil. Em caso de insucesso, o erro é enviado para a aplicação no momento que a mesma se reconectar.

3.4 Regras e Inferência

Para que uma aplicação possa utilizar os serviços oferecidos pelo eProfile, é necessário que a aplicação envie ao eProfile a ontologia de perfil que ele irá utilizar, extendendo a ontologia do eProfile. A ontologia é enviada quando a aplicação faz seu primeiro

acesso ao eProfile, e pode ser atualizada posteriormente. Ela deve ser enviada em formato OWL utilizando a sintaxe RDF/XML[15].

3.4.1 Regras de Inferência

A aplicação pode, a qualquer momento, criar, editar ou excluir regras para a inferência das informações de sua trilha, de modo a gerar informações úteis dentro da ontologia de perfil.

As regras devem ser enviadas em formato **SWRL**[16]. SWRL (*Semantic Web Rule Language*) é uma linguagem de regras proposta pela W3C. Esta linguagem foi escolhida pela expressividade que acrescenta à OWL.

Cada nova regra é gravada em uma base de regras. Assim, regras podem ser compartilhadas entre diferentes aplicações, desde que ambas extendam da mesma forma a ontologia de trilha.

Quando uma nova regra é enviada ao eProfile, ela deve conter as seguintes informações:

Nome: o nome da regra, para que possa ser atualizada ou excluída posteriormente;

Tipo de regra: se é uma regra em OWL, SPARQL ou SWRL;

Regra: a regra propriamente dita, na linguagem escolhida;

Objeto: classe, instância ou propriedade da ontologia de perfil que será atualizada;

Instrução de atualização: a forma como a instrução será atualizada;

Freqüência de atualização: a freqüência com que a regra deve ser atualizada;

Validade: a data limite até quando a regra será processada (pode ser ilimitada).

A instrução de atualização pode ser de um dos seguintes tipos:

Substituição: o item anterior é substituído por um novo item. Pode ser utilizado, por exemplo, em dados cadastrais como endereço e telefone, onde cada novo item substitui o anterior;

Adição: uma informação é adicionada, como a lista de endereços visitados por um carro;

Expressão: uma informação que precisa ser calculada. Neste caso, a própria aplicação passa a expressão do cálculo. Calcular o percentual de interesse de um usuário por um determinado assunto, por exemplo, seria uma informação deste tipo.

Uma vez que a regra tenha sido validada pelo agente regulador, ela entra em funcionamento e passa a ser elegível para execução.

As regras são executadas através de um motor de inferências externo. Foram estudados os seguintes raciocinadores que oferecem suporte a estas operações: Pellet[17], FaCT++[18] e HermiT[14]. Devido ao conjunto de funcionalidades e ao desempenho superior, o HermiT foi escolhido como o raciocinador utilizado pelo eProfile.

Um exemplo deste tipo de regra poderia ser a tentativa de identificação de parentesco entre duas entidades (neste caso, duas pessoas). Por exemplo, para identificar se duas pessoas são primas (isto é, se elas tem a propriedade `temPrimo` vinculando-as), a seguinte regra se aplicaria: `temPai(?a,?b)` $\wedge$ `temIrmao(?b,?c)` $\wedge$ `temFilho(?c,?d)` $\Rightarrow$ `temPrimo(?a,?d)`.

4 Validação

A validação deste modelo busca provar que o modelo apresentado é funcional e efetivo.

Para o teste de funcionalidade, o modelo será implementado e integrado com o eTrail, um gerenciador de trilhas. Uma vez que esta integração esteja realizada, um conjunto de regras será desenvolvido, de modo a verificar a capacidade de inferência do eProfile.

A seguir, será testada a capacidade do eProfile em gerenciar perfis de forma efetiva. Para tal, um grupo de usuários utilizará um aplicativo cujo perfil é gerenciado pelo eProfile. Após a utilização, será apresentado a estes usuários um questionário que buscará avaliar a efetividade do eProfile.

Referências

[1] M. Viviani, N. Bennani, and E. Egyed-Zsigmond, "A Survey on User Modeling in Multi-Application Environments," in *The Third International Conference on Advances in Human-oriented and*

Personalized Mechanisms, Technologies, and Services CENTRIC'10, pp. 111–116, IEEE, Aug. 2010.

[2] A. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 6, no. 1, pp. 4–7, 2001.

[3] M. Weiser, "The computer for the 21st century," *Scientific America*, pp. 94–104, 1991.

[4] M. Weiser and J. S. Brown, "Designing calm technology," *PowerGrid Journal*, July 1996.

[5] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo, "Content distribution in trial-aware environments," in *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, (New York, NY, USA), pp. 15:1–15:8, ACM, 2009.

[6] D. M. da Silva, J. Barbosa, and R. Vieira, "Octopus: um modelo para classificação de perfil de usuários usando trilhas em sistemas ubíquos," in *Companion Proceedings of the XIV Brazilian Symposium on Multimedia and the Web*, WebMedia '08, (New York, NY, USA), pp. 185–188, ACM, 2008.

[7] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje, "A Multi-Dimensional, Unified User Model for Cross-System Personalization," in *E4PIA Workshop 2004*, 2004.

[8] IEEE, "IEEE standard computer dictionary. a compilation of IEEE standard computer glossaries," *IEEE Std 610*, p. 1, 1991.

[9] F. Carmagnola and F. Cena, "User identification for cross-system personalisation," *Inf. Sci.*, vol. 179, pp. 16–32, January 2009.

[10] D. B. Lenat and R. V. Guha, *Building Large Knowledge-Based Systems; Representation and Inference in the Cyc Project*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1st ed., 1989.

[11] W3C, "RDF vocabulary description language 1.0: Rdf schema," 2011. `http://www.w3.org/TR/rdf-schema/`.

[12] M. C. Daconta, L. J. Obrst, and K. T. Smith, *The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management*. Indianapolis: Wiley, May 2003.

[13] G. D. Abowd, A. K. Dey, P. J. Brown, N. Davies, M. Smith, and P. Steggles, "Towards a better understanding of context and context-awareness," in *Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing*, HUC '99, (London, UK), pp. 304–307, Springer-Verlag, 1999.

[14] B. Motik, R. Shearer, and I. Horrocks, "Hypertableau Reasoning for Description Logics," *Journal of Artificial Intelligence Research*, vol. 36, pp. 165–228, 2009.

[15] W3C, "RDF/XML syntax specification (revised)," 2011. `http://www.w3.org/RDF/`.

[16] W3C, "SWRL: A semantic web rule language combining OWL and ruleml," 2004. `http://www.w3.org/Submission/SWRL/`.

[17] E. Sirin, B. Parsia, B. Grau, A. Kalyanpur, and Y. Katz, "Pellet: A practical OWL-DL reasoner," *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 5, pp. 51–53, June 2007.

[18] D. Tsarkov and I. Horrocks, "Fact++ description logic reasoner: System description," in *Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006)*, vol. 4130 of *Lecture Notes in Artificial Intelligence*, pp. 292–297, Springer, 2006.